PLAYHACK · SDE TRACK · IIT GUWAHATI

PLAYHACK

Campus sports facility booking where the winner of a race for one court is decided by Postgres, not by our code.

TEAM **INNOVAIT** INNOVAIT-HACKATHON.VERCEL.APP **0** OVERLAPPING BOOKINGS, EVER

THE PROBLEM, IN THE BRIEF'S OWN WORDS

AT 6 PM, FIFTY STUDENTS WANT THE SAME COURT

“The system must confirm **exactly one** and reject the rest, without corrupting data.”

Almost every booking system answers this the same way — and almost every one of them is wrong.

```
// the question
const taken = await isSlotTaken(court, slot);

// the answer
if (!taken) await insertBooking(court, slot);
```

Both lines are individually correct. Between them is a **gap** — and under load another request commits inside it.

THE ANSWER

THE WRITE *IS* THE DECISION

```
ALTER TABLE bookings ADD CONSTRAINT bookings_no_overlap
  EXCLUDE USING gist (facility_id WITH =, during WITH &&)
  WHERE (status = 'confirmed');
```

There is no question and no answer. A booking carries a time range, and the database **physically cannot store two overlapping ranges** for the same facility. The loser is rejected with SQLSTATE **23P01** before it ever becomes a row — no availability check on the write path, and no application lock on the slot.

WHY NOT A UNIQUE KEY

PARTIAL OVERLAP IS THE CASE THAT HAPPENS

	UNIQUE (FACILITY, STARTS_AT)	EXCLUDE … WITH &&
Two bookings at 18:00	**rejected**	**rejected**
18:00–19:00 vs 18:30–19:30	**both stored**	**rejected**
A two-hour closure over four slots	**not expressible**	**one row**
Cancelling frees the slot	needs a delete	**status flip**

A booking from 18:30 does not share a start time with one from 18:00 — but it **overlaps** it, and overlap is what matters to a student holding a racket.

PLAYHACK · IIT GUWAHATI · SDE TRACK

THE GAP BETWEEN THE QUESTION AND THE ANSWER

Fifty students tap Book on the same court at 18:00. The same burst, fired twice at the deployed app — once at a textbook availability check, once at a database that refuses to store the overlap.

```
ALTER TABLE bookings
  ADD CONSTRAINT bookings_no_overlap
  EXCLUDE USING gist (
    facility_id WITH =,
    during      WITH &&
  ) WHERE (status = 'confirmed');
```

1225

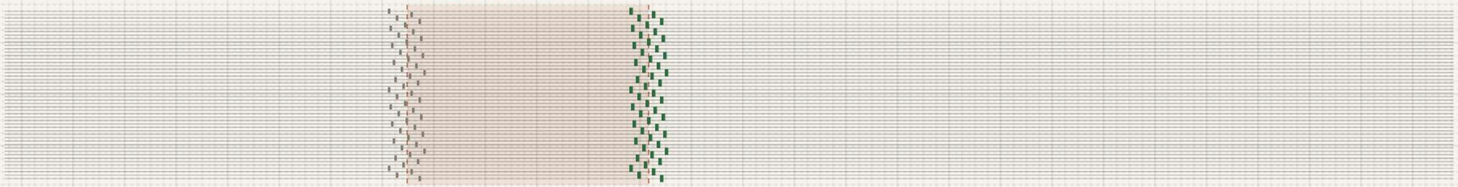
overlapping pairs left behind by **check-then-write**, swept across the whole table

0

from the identical burst, once the constraint is carrying the decision

01 CHECK, THEN WRITE

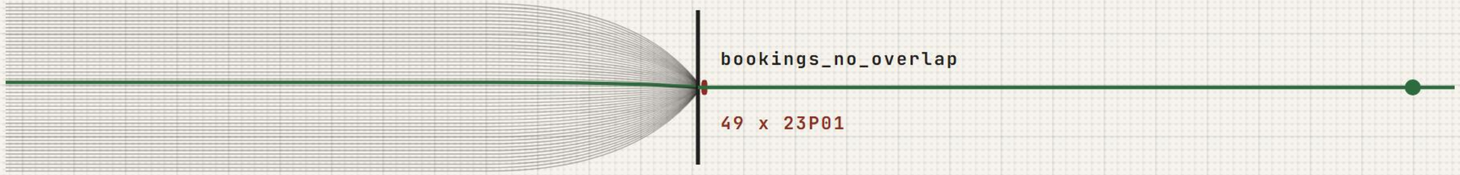
50 CONFIRMED · NOTHING ERRORED



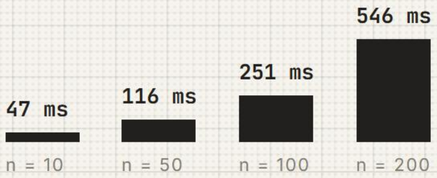
Every request reads "free", then writes. The shaded interval is the gap between the question and the answer — and fifty answers commit inside it.

02 THE WRITE IS THE DECISION

1 CONFIRMED · 49 REJECTED



No read. Fifty inserts race one exclusion constraint; Postgres rejects forty-nine with `SQLSTATE 23P01` before they can become rows.



Measured on the deployed app — Vercel **sin1**, Neon Postgres **ap-southeast-1**, warm, median of three. Server time is **sub-linear in n**: twenty times the load for ten times the time. The overlap count is a whole-table sweep after every run, not a check of the slot just tested.

THE SAME BURST, FIRED TWICE AT PRODUCTION

FIFTY REQUESTS. ONE COURT.

CHECK, THEN WRITE

50 bookings confirmed. Nothing errored.

1225 overlapping pairs, swept across the whole table

Fifty students walk to the same court.

THE WRITE IS THE DECISION

1 confirmed — 49 rejected with 23P01

0 overlapping pairs, across 380+ confirmed rows

Every rejection is typed and carries three alternative slots.

MEASURED ON THE DEPLOYED APP

SUB-LINEAR, NOT LINEAR

All contenders attempt the insert at once. One wins; the rest block on its uncommitted row and are released together the moment it commits. **Nothing in the write path serialises on n.**

Vercel **sin1**, Neon **ap-southeast-1**. Median of three warm runs.

MODE	REQUESTS	SERVER	CONFIRMED	OVERLAPS
naive	50	37 ms	50	1225
safe	10	47 ms	1	0
safe	50	116 ms	1	0
safe	100	251 ms	1	0
safe	**200**	**546 ms**	**1**	**0**

BEYOND “IT DOES NOT DOUBLE-BOOK”

CORRECTNESS IS THE FLOOR

Idempotency keys. A retried submit returns the original booking. A double-tap on hostel wifi is the common case, not the edge case.

Typed rejections. Losing returns `SLOT_TAKEN`, `QUOTA_EXCEEDED`, `OVERLAPS_OWN` — with three alternative slots and a queue position.

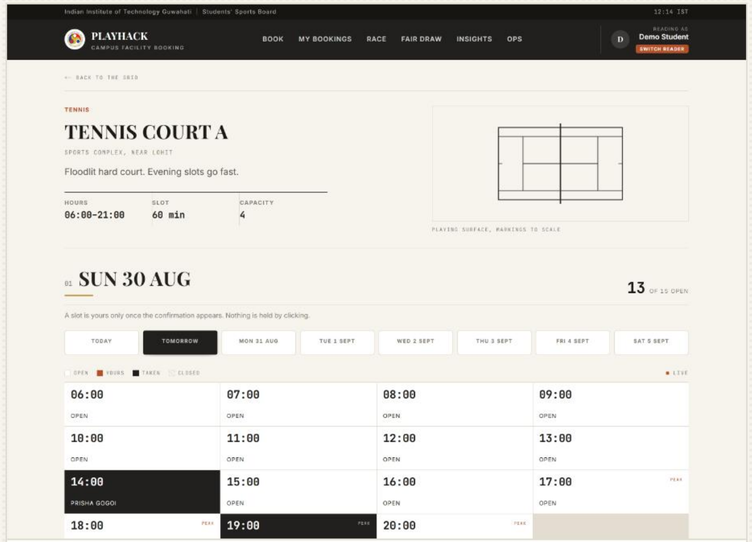
Waitlist promotion. Cancelling frees the slot *and* offers it to the next student in the same transaction — no window where it is free but unowned.

Maintenance closures. A closure is a booking row. One invariant, two features: a manager cannot close a court out from under a confirmed reservation.

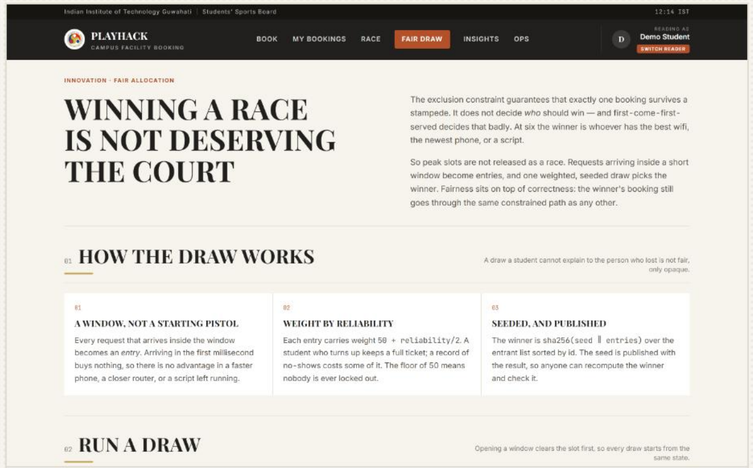
Fair draw. Peak slots go to a seeded, weighted, published lottery. First-come-first-served rewards the best wifi, not the keenest player.

THE PRODUCT

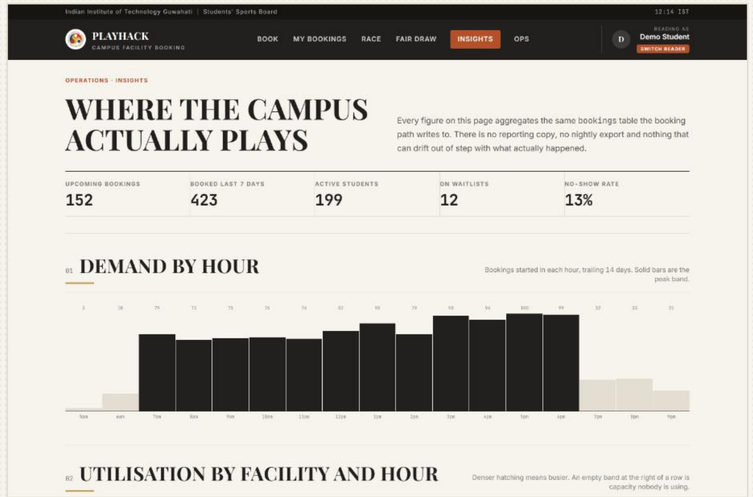
BUILT FOR A STUDENT ON A PHONE AT 5:58 PM



Booking. The day as a fixture table, live over SSE.



Fair draw. Rules and seed published before the draw.



Insights. Aggregated from the booking table itself.

VERIFIED, NOT ASSERTED

EVERY CLAIM ON THESE SLIDES IS REPRODUCIBLE

11/11 browser journey against production — book, confirm, cancel, slot returns

19 unit tests, run in CI against a real Postgres 17

0 npm audit vulnerabilities

```
npm run test        # concurrency + lottery
npm run invariant   # whole-table sweep
npm run e2e         # 11 browser checks
```

The overlap count is never a check of the slot just tested — it is **every confirmed row against every other row** on the same court, run after every race.

STATED PLAINLY

WHAT IS DELIBERATELY OUT OF SCOPE

No passwords. Identity is a signed cookie over a seeded roster, with a “switch reader” control — so a judge can race two students without two browser profiles. Swapping in SSO replaces one function.

The demo endpoints are unauthenticated and rate limited rather than gated, because the race demo has to stay one click away.

What is enforced: ownership scoped inside every UPDATE, zod on every request body, bind parameters on every query, no **`sql.unsafe`** anywhere.

Next: institute SSO, push notifications for waitlist offers, and a real limiter keyed on identity rather than on a header the client controls.

ONE PLATFORM. ZERO CLASHES.

FIFTY STUDENTS. ONE COURT. ONE WINNER.

Run the race yourself — the burst is fired at a real Postgres table through a real HTTP endpoint, and the whole table is swept afterwards.

innovait-hackathon.vercel.app/race

TEAM INNOVAIT · PLAYHACK SDE TRACK